



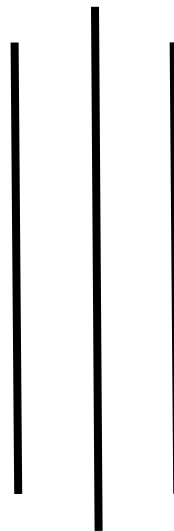
Tribhuwan University

Faculty of Humanities and Social Sciences

Bachelor of Computer Application

Mahendra Multiple Campus,

NEPALGUNJ, BANKE



**Lab Report of
Data Structure and Algorithms (CACS201)**

Submitted by:

Name: Jay Budha

Roll No :34

Submitted to:

Name: Pratik Chand

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my respected teacher Mr. Pratik Chand, for his continuous guidance, encouragement, and support throughout the course of this lab work on Data Structures and Algorithms (DSA). His expert teaching and insightful feedback have helped me develop a clear understanding of various data structures and their practical applications.

I am also thankful to the college management team BCA Faculty and Program Co-ordinator, **Mr Koshraj Regmi** for providing a well- structured academic environment and the opportunity to gain hands-on experience through this lab. Their efforts in organizing and facilitating the lab sessions are truly appreciated. Lastly, I would like to extend my heartfelt thanks to all my classmates, friends, and anyone who contributed, directly or indirectly, in helping me complete this project successfully and on time.

Yours sincerely,

Jay Budha

TABLE OF CONTENT

S.N	Lab Assignments	SIGNATURE
lab1:	WAP to implement the stack operations a. Create Stack b. Push Operation c. Pop Operation d. Display Data of Stack	
lab2	WAP to create linear queue and perform following operations: a. Enqueue Operation b. Dequeue Operation c. Display Operation	
lab3	WAP to create circular queue and perform following operations a. Enqueue Operation b. Dequeue Operation c. Display Operation	
lab4	WAP to create double ended queue and perform following operations a. Enqueue Operation b. Dequeue Operation c. Display Operation	
lab5	WAP to create singly linked list and perform following operations a. Insert at first b. Insert at last c. Insert before specific position d. Insert after specific position e. Insert at specific position f. Delete at first g. Delete at last h. Delete at specific position i. Delete the node having specific value j. Display the data of singly linked list	
lab6	WAP to create doubly linked list and perform following operations a. Insert at first b. Insert at last c. Insert before specific position d. Insert after specific position e. Insert at specific position f. Delete at first g. Delete at last h. Delete at specific position i. Delete the node having specific value j. Display the data of singly linked list	

lab7	WAP to create circular singly linked list and perform following operations a. Insert at first b. Insert at last c. Insert before specific position d. Insert after specific position e. Insert at specific position f. Delete at first g. Delete at last h. Delete at specific position i. Delete the node having specific value j. Display the data of singly linked list	
lab8	Lab 8: WAP shift the 3 deices from 1st tower to 3rd tower in Tower of Hanoi using recursion.	
lab9	WAP to find GCD of any two numbers using recursion.	
lab10	WAP to implement the bubble sort algorithm.	
lab11	WAP to implement the insertion sort algorithm.	
lab12	WAP to implement the merge sort algorithm.	
lab13	WAP to implement the quick sort algorithm.	
lab14	WAP to implement the selection sort algorithm.	
lab15	WAP to implement the shell sort algorithm.	
lab16	WAP to implement the heap sort algorithm.	
lab17	WAP to implement sequential searching algorithm.	
lab18	WAP to implement binary searching algorithm.	
lab19	WAP to create binary search tree and perform following operations a. Insertion Operation b. Deletion Operation c. Search data from BST d. In-order traversal e. Pre-order traversal f. Post-order traversal	
lab20	WAP to implement Dijkstra's Shortest Path Algorithm.	

Lab 1: WAP to implement the stack operations

- a. Create Stack
- b. Push Operation
- c. Pop Operation
- d. Display Data of Stack

```
public class Stack {
    int[] mystack;
    int top;
    public Stack(int size) {
        mystack = new int[size];
        top = -1;
        System.out.println("Stack is created");
    }
    // Push data into the stack
    public void push(int data) {
        if (top == mystack.length - 1) {
            System.out.println("Error: Stack overflow.");
            return;
        }
        top = top + 1;
        mystack[top] = data;
        System.out.println("Data " + data + " is pushed into the stack.");
    }
    // Pop data from stack
    public void pop() {
        if (top == -1) {
            System.out.println("Error: Stack underflow.");
            return;
        }
        int data = mystack[top];
        top = top - 1;
        System.out.println("Data popped from stack is " + data);
    }
    // Display data remaining in stack
    public void display() {
        if (top == -1) {
            System.out.println("Stack is empty");
        }
        else
        {
            System.out.println("Data Remaining in stack:");
        }
    }
}
```

```

        for (int i = top; i >= 0; i--) {
            System.out.print(mystack[i] + " ");
        }
    }
}

public static void main(String[] args) {
    Stack newStack = new Stack(5);
    newStack.push(10);
    newStack.push(20);
    newStack.push(30);
    newStack.push(20);
    newStack.push(30);
    newStack.push(100);
    newStack.pop();
    newStack.pop();
    newStack.pop();
    newStack.display();
}
}

```

Output:

Stack is created.

Data 10 is pushed into the stack.

Data 20 is pushed into the stack.

Data 30 is pushed into the stack.

Data 20 is pushed into the stack.

Data 30 is pushed into the stack.

Error: Stack overflow.

Data popped from stack is 30

Data popped from stack is 20

Data popped from stack is 30

Data remaining in stack: 20 10

Lab 2: WAP to create linear queue and perform following operations:

- a. Enqueue Operation
- b. Dequeue Operation
- c. Display Operation

```
public class LinearQueue {
    int[] linearQueue;
    int front;
    int rear;
    // Constructor
    public LinearQueue(int size) {
        linearQueue = new int[size];
        front = -1;
        rear = -1;
    }
    // Enqueue operation
    public void enqueue(int data) {
        if (front == -1 && rear == -1) {
            front = 0;
        }
        if (rear == linearQueue.length - 1) {
            System.out.println("Queue is overflow");
            return;
        }
        rear = rear + 1;
        linearQueue[rear] = data;
        System.out.println("Data " + data + " is inserted in linear queue");
    }
    // Dequeue operation
    public void dequeue() {
        if (front == -1 || front > rear) {
            System.out.println("Queue is empty");
        } else {
            int item = linearQueue[front];
            front = front + 1;
            System.out.println("Data " + item + " is deleted from queue");
        }
    }
    // Display operation
    public void display() {
        if (front == -1 || front > rear) {
            System.out.println("Queue is empty");
        }
    }
}
```

```

    } else {
        System.out.print("Data in queue: ");
        for (int i = front; i <= rear; i++) {
            System.out.print(linearQueue[i] + " ");
        }
        System.out.println();
    }
}

public static void main(String[] args) {
    LinearQueue linearQueue = new LinearQueue(5);
    linearQueue.display();
    linearQueue.dequeue();
    linearQueue.enqueue(10);
    linearQueue.enqueue(20);
    linearQueue.enqueue(30);
    linearQueue.enqueue(10);
    linearQueue.enqueue(20);
    linearQueue.enqueue(30); // Overflow
    linearQueue.dequeue();
    linearQueue.dequeue();
    linearQueue.dequeue();
    linearQueue.dequeue();
    linearQueue.display();
    linearQueue.enqueue(50);
}
}

```

Output:

Queue is empty
 Data 10 is inserted in linear queue
 Data 20 is inserted in linear queue
 Data 30 is inserted in linear queue
 Data 10 is inserted in linear queue
 Data 20 is inserted in linear queue
 Queue is overflow
 Data 10 is deleted from queue
 Data 20 is deleted from queue
 Data 30 is deleted from queue
 Data 10 is deleted from queue
 Data in queue: 20
 Data 50 is inserted in linear queue

Lab 3: WAP to create circular queue and perform following operations

- a. Enqueue Operation
- b. Dequeue Operation
- c. Display Operation

```
public class CircularQueue {
    int[] cque;
    int front, rear;
    int deletedData;
    // Constructor
    CircularQueue(int size) {
        cque = new int[size];
        front = rear = -1;
    }
    // Check if full
    boolean isFull() {
        return (front == 0 && rear == cque.length - 1) || (front == rear + 1);
    }
    // Check if empty
    boolean isEmpty() {
        return front == -1 && rear == -1;
    }
    // Enqueue operation
    void enqueue(int data) {
        if (isFull()) {
            System.out.println("Queue is full");
        } else if (isEmpty()) {
            front = rear = 0;
            cque[rear] = data;
            System.out.println("Data " + data + " is inserted at index " + rear);
        }
        else if (rear == cque.length - 1 && front != 0)
        {
            rear = 0;
            cque[rear] = data;
            System.out.println("Data " + data + " is inserted at index " + rear + "
(wrapped)");
        } else {
            rear++;
            cque[rear] = data;
            System.out.println("Data " + data + " is inserted at index " + rear);
        }
    }
}
```

```

}
// Dequeue operation
void dequeue() {
    if (isEmpty()) {
        System.out.println("Queue is empty");
    } else if (front == rear) {
        deletedData = cque[front];
        System.out.println("Data " + deletedData + " is deleted from index " + front);
        front = rear = -1;
    } else if (front == cque.length - 1) {
        deletedData = cque[front];
        System.out.println("Data " + deletedData + " is deleted from index " + front);
        front = 0;
    } else {
        deletedData = cque[front];
        System.out.println("Data " + deletedData + " is deleted from index " + front);
        front++;
    }
}

// Display operation
void display() {
    if (isEmpty()) {
        System.out.println("Queue is empty");
    } else {
        System.out.println("Data in queue:");
        int i = front;
        while (i != rear) {
            System.out.print(cque[i] + " ");
            i = (i + 1) % cque.length;
        }
        System.out.println(cque[rear]);
    }
}

// Peek front
void peekFront() {
    if (isEmpty()) {
        System.out.println("Front is at index -1");
    } else {
        System.out.println("Front is at index " + front + " (Data: " + cque[front] + ")");
    }
}

```

```

// Peek rear
void peekRear() {
    if (isEmpty()) {
        System.out.println("Rear is at index -1");
    } else {
        System.out.println("Rear is at index " + rear + " (Data: " + cque[rear] + ")");
    }
}

public static void main(String[] args) {
    CircularQueue cque = new CircularQueue(4);
    cque.enqueue(10);
    cque.enqueue(20);
    cque.enqueue(30);
    cque.enqueue(40);
    cque.enqueue(50); // Queue is full
    cque.display();
    cque.dequeue();
    cque.dequeue();
    cque.display();
    cque.peekFront();
    cque.peekRear();
}
}

```

Output:

```

Data 10 is inserted at index 0
Data 20 is inserted at index 1
Data 30 is inserted at index 2
Data 40 is inserted at index 3
Queue is full
Data in queue:
10 20 30 40
Data 10 is deleted from index 0
Data 20 is deleted from index 1
Data in queue:
30 40
Front is at index 2 (Data: 30)
Rear is at index 3 (Data: 40)

```

Lab 4: WAP to create double ended queue and perform operations

- a. Insert at front
- b. Insert at rear
- c. Delete from front
- d. Delete from rear
- e. Display

```
public class DoubleEndedQueue {
    int[] deque;
    int front, rear, size;
    // Constructor
    public DoubleEndedQueue(int size) {
        deque = new int[size];
        front = -1;
        rear = -1;
        this.size = size;
    }
    // Check if deque is full
    boolean isFull() {
        return (front == 0 && rear == size - 1) || (front == rear + 1);
    }
    // Check if deque is empty
    boolean isEmpty() {
        return front == -1;
    }
    // Insert at front
    void insertFront(int data) {
        if (isFull()) {
            System.out.println("Deque is full");
            return;
        }
        if (isEmpty()) {
            front = rear = 0;
        } else if (front == 0) {
            front = size - 1;
        } else {
            front--;
        }
        deque[front] = data;
        System.out.println(data + " inserted at front (Index: " + front + ")");
    }
}
```

// Insert at rear

```
void insertRear(int data) {  
    if (isFull()) {  
        System.out.println("Deque is full");  
        return;  
    }  
    if (isEmpty()) {  
        front = rear = 0;  
    } else if (rear == size - 1) {  
        rear = 0;  
    } else {  
        rear++;  
    }  
    deque[rear] = data;  
    System.out.println(data + " inserted at rear (Index: " + rear + ")");  
}
```

// Delete from front

```
void deleteFront() {  
    if (isEmpty()) {  
        System.out.println("Deque is empty");  
        return;  
    }  
    int data = deque[front];  
    if (front == rear) {  
        front = rear = -1;  
    } else if (front == size - 1) {  
        front = 0;  
    } else {  
        front++;  
    }  
    System.out.println(data + " deleted from front");  
}
```

// Delete from rear

```
void deleteRear() {  
    if (isEmpty()) {  
        System.out.println("Deque is empty");  
        return;  
    }  
    int data = deque[rear];  
    if (front == rear) {  
        front = rear = -1;  
    } else if (rear == 0) {
```

```

        rear = size - 1;
    } else {
        rear--;
    }
    System.out.println(data + " deleted from rear");
}
// Display deque
void display() {
    if (isEmpty()) {
        System.out.println("Deque is empty");
        return;
    }
    System.out.print("Deque: ");
    int i = front;
    while (true) {
        System.out.print(deque[i] + " ");
        if (i == rear) break;
        i = (i + 1) % size;
    }
    System.out.println();
}
public static void main(String[] args) {
    DoubleEndedQueue dq = new DoubleEndedQueue(5);
    dq.insertRear(10);
    dq.insertRear(20);
    dq.insertFront(5);
    dq.insertFront(1);
    dq.display();
    dq.deleteFront();
    dq.deleteRear();
    dq.display();
    dq.insertRear(30);
    dq.insertRear(40);
    dq.insertFront(0);
    dq.display();
    dq.insertRear(50); // Deque full
}
}

```

Output:

10 inserted at rear (Index: 0)

20 inserted at rear (Index: 1)

5 inserted at front (Index: 4)

1 inserted at front (Index: 3)

Deque: 1 5 10 20

1 deleted from front

20 deleted from rear

Deque: 5 10

30 inserted at rear (Index: 2)

40 inserted at rear (Index: 3)

0 inserted at front (Index: 2)

Deque: 0 5 10 30 40

Deque is full

Lab 5: WAP to create singly linked list and perform operations

- a. Insert at first
- b. Insert at last
- c. Insert before position
- d. Insert after position
- e. Insert at position
- f. Delete first
- g. Delete last
- h. Delete at position
- i. Delete by value
- j. Display

```
public class SLLOperations {
    static class Node {
        int data;
        Node next;
        Node head;
        Node() {
            head = null;
        }
        // Insert at first
        void insertFirst(int item) {
            Node newNode = new Node();
            newNode.data = item;
            if (head == null) {
                head = newNode;
                return;
            }
            newNode.next = head;
            head = newNode;
        }
        // Insert at last
        void insertLast(int item) {
            Node newNode = new Node();
            newNode.data = item;
            if (head == null) {
                head = newNode;
                return;
            }
            Node temp = head;
            while (temp.next != null) {
```

```

        temp = temp.next;
    }
    temp.next = newNode;
}
// Display list
void display() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }
    Node temp = head;
    System.out.print("SLL: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
    System.out.println();
}
// Delete first
void deleteFirst() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }
    head = head.next;
}
// Delete last
void deleteLast() {
    if (head == null) return;
    if (head.next == null) {
        head = null;
        return;
    }
    Node temp = head;
    while (temp.next.next != null) {
        temp = temp.next;
    }
    temp.next = null;
}
public static void main(String[] args) {
    Node sll = new Node();
    System.out.println("=== Singly Linked List Operations ===");
}

```

```
// Insertions
sll.insertFirst(30);
sll.insertFirst(20);
sll.insertFirst(10);
System.out.print("After inserting 10,20,30 at front: ");
sll.display();
sll.insertLast(40);
sll.insertLast(50);
System.out.print("After inserting 40,50 at end: ");
sll.display();
// Deletions
sll.deleteFirst();
System.out.print("After deleting first: ");
sll.display();
sll.deleteLast();
System.out.print("After deleting last: ");
sll.display();
    }
}
}
```

Output:

After inserting 10,20,30 at front: SLL: 10 20 30
After inserting 40,50 at end: SLL: 10 20 30 40 50
After deleting first: SLL: 20 30 40 50
After deleting last: SLL: 20 30 40

Lab 6: WAP to create doubly linked list and perform operations

- a. Insert at first
- b. Insert at last
- c. Insert before position
- d. Insert after position
- e. Delete first
- f. Delete last
- g. Delete at position
- h. Delete by value
- i. Display (forward & backward)

```
public class DLLOperations {
    static class Node {
        int data;
        Node prev, next;
        Node head;
        Node tail;
        Node() {
            head = tail = null;
        }
        // Insert at first
        void insertFirst(int item) {
            Node newNode = new Node();
            newNode.data = item;
            if (head == null) {
                head = tail = newNode;
                return;
            }
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
            System.out.println(item + " inserted at front");
        }
        // Insert at last
        void insertLast(int item) {
            Node newNode = new Node();
            newNode.data = item;
            if (tail == null) {
                head = tail = newNode;
                return;
            }
            tail.next = newNode;
```

```

        newNode.prev = tail;
        tail = newNode;
        System.out.println(item + " inserted at end");
    }
    // Display forward
    void displayForward() {
        if (head == null) {
            System.out.println("List is empty");
            return;
        }
        Node temp = head;
        System.out.print("Forward: ");
        while (temp != null) {
            System.out.print(temp.data + " ");
            temp = temp.next;
        }
        System.out.println();
    }
    // Display backward
    void displayBackward() {
        if (tail == null) {
            System.out.println("List is empty");
            return;
        }
        Node temp = tail;
        System.out.print("Backward: ");
        while (temp != null) {
            System.out.print(temp.data + " ");
            temp = temp.prev;
        }
        System.out.println();
    }
    // Delete first
    void deleteFirst() {
        if (head == null) {
            System.out.println("List is empty");
            return;
        }
        System.out.println(head.data + " deleted from front");
        if (head == tail) {
            head = tail = null;
            return;
        }
    }

```

```

    }
    head = head.next;
    head.prev = null;
}
// Delete last
void deleteLast() {
    if (tail == null) {
        System.out.println("List is empty");
        return;
    }
    System.out.println(tail.data + " deleted from end");
    if (head == tail) {
        head = tail = null;
        return;
    }
    tail = tail.prev;
    tail.next = null;
}
public static void main(String[] args) {
    Node dll = new Node();
    System.out.println("Doubly Linked List Operations ");
    // Insertions
    dll.insertFirst(30);
    dll.insertFirst(20);
    dll.insertFirst(10);
    dll.displayForward();
    dll.displayBackward();
    dll.insertLast(40);
    dll.insertLast(50);
    dll.displayForward();
    // Deletions
    dll.deleteFirst();
    dll.displayForward();
    dll.deleteLast();
    dll.displayForward();
    dll.displayBackward();
}
}
}

```

Output:

Doubly Linked List Operations

10 inserted at front

20 inserted at front

30 inserted at front

Forward: 30 20 10

Backward: 10 20 30

40 inserted at end

50 inserted at end

Forward: 30 20 10 40 50

30 deleted from front

Forward: 20 10 40 50

50 deleted from end

Forward: 20 10 40

Backward: 40 10 20

Lab 7: WAP to create circular singly linked list and perform following operations

- a. Insert at first
- b. Insert at last
- c. Insert before specific position
- d. Insert after specific position
- e. Insert at specific position
- f. Delete at first
- g. Delete at last
- h. Delete at specific position
- i. Delete the node having specific value
- j. Display the data of singly linked list

```
public class CSLLOperations {
    static class Node {
        int data;
        Node next;
        Node last; // acts as tail pointer
        Node() {
            last = null;
        }
        // Insert at first
        void insertFirst(int item) {
            Node newNode = new Node();
            newNode.data = item;
            if (last == null) {
                last = newNode;
                newNode.next = newNode;
                System.out.println(item + " inserted as first node");
                return;
            }
            newNode.next = last.next;
            last.next = newNode;
            System.out.println(item + " inserted at front");
        }
        // Insert at last
        void insertLast(int item) {
            Node newNode = new Node();
            newNode.data = item;
            if (last == null) {
                last = newNode;
                newNode.next = newNode;
            }
        }
    }
}
```

```

        System.out.println(item + " inserted as first node");
        return;
    }
    newNode.next = last.next;
    last.next = newNode;
    last = newNode;
    System.out.println(item + " inserted at end");
}
// Display list
void display() {
    if (last == null) {
        System.out.println("List is empty");
        return;
    }
    Node temp = last.next;
    System.out.print("CSLL: ");
    do {
        System.out.print(temp.data + " ");
        temp = temp.next;
    } while (temp != last.next);
    System.out.println();
}
// Delete first
void deleteFirst() {
    if (last == null) {
        System.out.println("List is empty");
        return;
    }
    if (last.next == last) {
        System.out.println(last.data + " deleted (last node)");
        last = null;
        return;
    }
    Node first = last.next;
    last.next = first.next;
    System.out.println(first.data + " deleted from front");
}
// Delete last
void deleteLast() {
    if (last == null) {
        System.out.println("List is empty");
        return;
    }

```

```

    }
    if (last.next == last) {
        System.out.println(last.data + " deleted (last node)");
        last = null;
        return;
    }
    Node temp = last.next;
    while (temp.next != last) {
        temp = temp.next;
    }
    System.out.println(last.data + " deleted from end");
    temp.next = last.next;
    last = temp;
}

public static void main(String[] args) {
    Node csll = new Node();
    System.out.println("=== Circular Singly Linked List Operations ===");
    // Insertions
    csll.insertFirst(30);
    csll.insertFirst(20);
    csll.insertFirst(10);
    csll.display();
    csll.insertLast(40);
    csll.insertLast(50);
    csll.display();
    // Deletions
    csll.deleteFirst();
    csll.display();
    csll.deleteLast();
    csll.display();
    // Circular verification
    System.out.println("Checking circular reference:");
    System.out.println("Last node points to: " + csll.last.next.data);
}
}
}

```

Output:

30 inserted as first node

20 inserted at front

10 inserted at front

CSLL: 10 20 30

40 inserted at end

50 inserted at end

CSLL: 10 20 30 40 50

10 deleted from front

CSLL: 20 30 40 50

50 deleted from end

CSLL: 20 30 40

Checking circular reference:

Last node points to: 20

Lab 8: WAP shift the 3 deices from 1st tower to 3rd tower in Tower of Hanoi using recursion.

```
public class TowerOfHanoi {  
    public static void solveHanoi(int disks, char source, char auxiliary, char  
destination) {  
        if (disks == 1) {  
            System.out.println("Move disk 1 from " + source + " to " + destination);  
            return;  
        }  
        solveHanoi(disks - 1, source, destination, auxiliary);  
        System.out.println("Move disk " + disks + " from " + source + " to " +  
destination);  
        solveHanoi(disks - 1, auxiliary, source, destination);  
    }  
    public static void main(String[] args) {  
        int disks = 3;  
        System.out.println("Tower of Hanoi solution for " + disks + " disks:");  
        solveHanoi(disks, 'A', 'B', 'C');  
    }  
}
```

Output:

Tower of Hanoi solution for 3 disks:

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

Lab 9: WAP to find GCD of any two numbers using recursion.

```
public class GCDCalculator {  
    public static int findGCD(int a, int b) {  
        if (b == 0) {  
            return a;  
        }  
        return findGCD(b, a % b);  
    }  
    public static void main(String[] args) {  
        int num1 = 48, num2 = 18;  
        System.out.println("GCD of " + num1 + " and " + num2 + " is: " + findGCD(num1,  
num2));  
    }  
}
```

Output:

GCD of 48 and 18 is: 6

Lab 10: WAP to implement the bubble sort algorithm.

```
public class BubbleSort {
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n-1; i++) {
            boolean swapped = false;
            for (int j = 0; j < n-i-1; j++) {
                if (arr[j] > arr[j+1]) {
                    // Swap elements
                    int temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                    swapped = true;
                }
            }
            if (!swapped) break; // Optimization for already sorted arrays
        }
    }
    public static void main(String[] args) {
        int[] data = {64, 34, 25, 12, 22, 11, 90};
        System.out.println("Original array: " + java.util.Arrays.toString(data));

        bubbleSort(data);
        System.out.println("Sorted array: " + java.util.Arrays.toString(data));
    }
}
```

Output:

Original array: [64, 34, 25, 12, 22, 11, 90]

Sorted array: [11, 12, 22, 25, 34, 64, 90]

Lab 11: WAP to implement the insertion sort algorithm.

```
public class InsertionSort {
    public static void insertionSort(int[] arr) {
        int n = arr.length;
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;

            /* Shift elements greater than key to the right */
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }

    public static void main(String[] args) {
        int[] data = {12, 11, 13, 5, 6};
        System.out.println("Original array: " + Arrays.toString(data));

        insertionSort(data);
        System.out.println("Sorted array: " + Arrays.toString(data));
    }
}
```

Output:

Original array: [12, 11, 13, 5, 6]

Sorted array: [5, 6, 11, 12, 13]

Lab 12: WAP to implement the merge sort algorithm.

```
public class MergeSort {
    public static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }
    private static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;
        int[] L = new int[n1];
        int[] R = new int[n2];
        System.arraycopy(arr, left, L, 0, n1);
        System.arraycopy(arr, mid + 1, R, 0, n2);
        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) {
                arr[k++] = L[i++];
            } else {
                arr[k++] = R[j++];
            }
        }
        while (i < n1) arr[k++] = L[i++];
        while (j < n2) arr[k++] = R[j++];
    }
    public static void main(String[] args) {
        int[] data = {38, 27, 43, 3, 9, 82, 10};
        System.out.println("Original array: " + Arrays.toString(data));
        mergeSort(data, 0, data.length - 1);
        System.out.println("Sorted array: " + Arrays.toString(data));
    }
}
```

Output:

Original array: [38, 27, 43, 3, 9, 82, 10]

Sorted array: [3, 9, 10, 27, 38, 43, 82]

Lab 13: WAP to implement the quick sort algorithm.

```
public class QuickSort {
    public static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }
    private static int partition(int[] arr, int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                swap(arr, i, j);
            }
        }
        swap(arr, i + 1, high);
        return i + 1;
    }
    private static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
    public static void main(String[] args) {
        int[] data = {10, 7, 8, 9, 1, 5};
        System.out.println("Original array: " + Arrays.toString(data));
        quickSort(data, 0, data.length - 1);
        System.out.println("Sorted array: " + Arrays.toString(data));
    }
}
```

Output:

Original array: [10, 7, 8, 9, 1, 5]

Sorted array: [1, 5, 7, 8, 9, 10]

Lab 14: WAP to implement the selection sort algorithm.

```
public class SelectionSort {  
    public static void main(String[] args) {  
        int[] arr = {29, 10, 14, 37, 13};  
        for (int i = 0; i < arr.length - 1; i++) {  
            int minIdx = i;  
            for (int j = i + 1; j < arr.length; j++)  
                if (arr[j] < arr[minIdx])  
                    minIdx = j;  
            int temp = arr[i];  
            arr[i] = arr[minIdx];  
            arr[minIdx] = temp;  
        }  
        System.out.println("Sorted array:");  
        for (int num : arr)  
            System.out.print(num + " ");  
    }  
}
```

Output:

Sorted array:
10 13 14 29 37

Lab 15: WAP to implement the shell sort algorithm.

```
public class ShellSort {  
    public static void main(String[] args) {  
        int[] arr = {64, 34, 25, 12, 22, 11, 90};  
        int n = arr.length;  
  
        for (int gap = n / 2; gap > 0; gap /= 2) {  
            for (int i = gap; i < n; i++) {  
                int temp = arr[i];  
                int j;  
                for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)  
                    arr[j] = arr[j - gap];  
                arr[j] = temp;  
            }  
        }  
        System.out.println("Sorted array:");  
        for (int num : arr)  
            System.out.print(num + " ");  
    }  
}
```

Output:

Sorted array:

11 12 22 25 34 64 90

Lab 16: WAP to implement the heap sort algorithm.

```
public class HeapSort {
    public static void heapify(int arr[], int n, int i) {
        int largest = i, left = 2 * i + 1, right = 2 * i + 2;
        if (left < n && arr[left] > arr[largest])
            largest = left;
        if (right < n && arr[right] > arr[largest])
            largest = right;
        if (largest != i) {
            int swap = arr[i];
            arr[i] = arr[largest];
            arr[largest] = swap;
            heapify(arr, n, largest);
        }
    }
    public static void heapSort(int arr[]) {
        int n = arr.length;
        for (int i = n / 2 - 1; i >= 0; i--)
            heapify(arr, n, i);
        for (int i = n - 1; i >= 0; i--) {
            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;
            heapify(arr, i, 0);
        }
    }
    public static void main(String args[]) {
        int arr[] = {12, 11, 13, 5, 6, 7};
        heapSort(arr);
        System.out.println("Sorted array:");
        for (int num : arr)
            System.out.print(num + " ");
    }
}
```

Output:

Sorted array:
5 6 7 11 12 13

Lab 17: WAP to implement sequential searching algorithm.

```
public class SequentialSearch {  
    public static void main(String[] args) {  
        int[] arr = {12, 34, 10, 3, 22};  
        int key = 10;  
        boolean found = false;  
  
        for (int i = 0; i < arr.length; i++) {  
            if (arr[i] == key) {  
                System.out.println("Element found at index: " + i);  
                found = true;  
                break;  
            }  
        }  
        if (!found)  
            System.out.println("Element not found");  
    }  
}
```

Output:

Element found at index: 2

Lab 18: WAP to implement binary searching algorithm.

```
import java.util.Arrays;

public class BinarySearch {
    public static void main(String[] args) {
        int[] arr = {3, 10, 12, 22, 34};
        int key = 22;
        int low = 0, high = arr.length - 1, mid;
        boolean found = false;
        while (low <= high) {
            mid = (low + high) / 2;
            if (arr[mid] == key) {
                System.out.println("Element found at index: " + mid);
                found = true;
                break;
            } else if (arr[mid] < key)
                low = mid + 1;
            else
                high = mid - 1;
        }
        if (!found)
            System.out.println("Element not found");
    }
}
```

Output:

Element found at index: 3

Lab 19: WAP to create binary search tree and perform following operations

- a. Insertion Operation
- b. Deletion Operation
- c. Search data from BST
- d. In-order traversal
- e. Pre-order traversal
- f. Post-order traversal

```
class Node {
    int key;
    Node left, right;
    Node(int item) {
        key = item;
        left = right = null;
    }
}
class BST {
    Node root;
    BST() {
        root = null;
    }
    void insert(int key) {
        root = insertRec(root, key);
    }
    Node insertRec(Node root, int key) {
        if (root == null) return new Node(key);
        if (key < root.key)
            root.left = insertRec(root.left, key);
        else if (key > root.key)
            root.right = insertRec(root.right, key);
        return root;
    }
    void inorder(Node root) {
        if (root != null) {
            inorder(root.left);
            System.out.print(root.key + " ");
            inorder(root.right);
        }
    }
    void preorder(Node root) {
        if (root != null) {
```

```

        System.out.print(root.key + " ");
        preorder(root.left);
        preorder(root.right);
    }
}
void postorder(Node root) {
    if (root != null) {
        postorder(root.left);
        postorder(root.right);
        System.out.print(root.key + " ");
    }
}
Node search(Node root, int key) {
    if (root == null || root.key == key)
        return root;
    if (key < root.key)
        return search(root.left, key);
    return search(root.right, key);
}
Node delete(Node root, int key) {
    if (root == null) return root;
    if (key < root.key)
        root.left = delete(root.left, key);
    else if (key > root.key)
        root.right = delete(root.right, key);
    else {
        if (root.left == null)
            return root.right;
        else if (root.right == null)
            return root.left;
        root.key = minValue(root.right);
        root.right = delete(root.right, root.key);
    }
    return root;
}
int minValue(Node root) {
    int minVal = root.key;
    while (root.left != null) {
        minVal = root.left.key;
        root = root.left;
    }
    return minVal;
}

```

```

    }
}
public class BSTOperations {
    public static void main(String[] args) {
        BST tree = new BST();
        int[] keys = {50, 30, 70, 20, 40, 60, 80};
        for (int key : keys)
            tree.insert(key);
        System.out.print("Inorder traversal: ");
        tree.inorder(tree.root);
        System.out.println("\nPreorder traversal: ");
        tree.preorder(tree.root);
        System.out.println("\nPostorder traversal: ");
        tree.postorder(tree.root);
        int searchKey = 40;
        System.out.println("\n\nSearching for " + searchKey + ":");
        System.out.println(tree.search(tree.root, searchKey) != null ? "Found" : "Not
Found");
        System.out.println("\nDeleting 70...");
        tree.root = tree.delete(tree.root, 70);
        System.out.print("Inorder after deletion: ");
        tree.inorder(tree.root);
    }
}

```

Output:

```

Inorder traversal: 20 30 40 50 60 70 80
Preorder traversal:
50 30 20 40 70 60 80
Postorder traversal:
20 40 30 60 80 70 50
Searching for 40:
Found
Deleting 70...
Inorder after deletion: 20 30 40 50 60 80

```

Lab 20: WAP to implement Dijkstra's Shortest Path Algorithm.

```
public class Dijkstra {
    static final int V = 6;
    int minDistance(int dist[], boolean visited[]) {
        int min = Integer.MAX_VALUE, minIndex = -1;
        for (int v = 0; v < V; v++)
            if (!visited[v] && dist[v] <= min)
        {
            min = dist[v];
            minIndex = v;
        }

        return minIndex;
    }

    void dijkstra(int graph[][], int src) {
        int dist[] = new int[V];
        boolean visited[] = new boolean[V];

        for (int i = 0; i < V; i++) {
            dist[i] = Integer.MAX_VALUE;
            visited[i] = false;
        }

        dist[src] = 0;
        for (int count = 0; count < V - 1; count++) {
            int u = minDistance(dist, visited);
            visited[u] = true;
            for (int v = 0; v < V; v++)
                if (!visited[v] && graph[u][v] != 0 &&
                    dist[u] != Integer.MAX_VALUE &&
                    dist[u] + graph[u][v] < dist[v])
                    dist[v] = dist[u] + graph[u][v];
        }

        System.out.println("Vertex \t Distance from Source");
        for (int i = 0; i < V; i++)
            System.out.println(i + " \t " + dist[i]);
    }

    public static void main(String[] args) {
        int graph[][] = new int[V][V] {
```

```
        {0, 4, 0, 0, 0, 0},  
        {4, 0, 8, 0, 0, 0},  
        {0, 8, 0, 7, 0, 4},  
        {0, 0, 7, 0, 9, 14},  
        {0, 0, 0, 9, 0, 10},  
        {0, 0, 4, 14, 10, 0}  
    };  
    Dijkstra d = new Dijkstra();  
    d.dijkstra(graph, 0);  
}  
}
```

Output:

Vertex	Distance from Source
0	0
1	4
2	12
3	19
4	21
5	16